지능형 DB 12주

PL/SQL (개요, PROCEDURE)

PL/SQL 개요

Procedure Language

비절차적 언어인 SQL에 프로그래밍적 요소(예, 변수사용 등)를 가미해서 좀 더 다이나믹하게 프로그래밍 할 수 있도록 하는 기능

엑셀의 매크로와 비슷한 기능

일단 한번 만 구현하면 여러 번 반복 사용가능

한 건만 처리하는 'procedure'와 여러 건을 처리하는 'cursor'로 구분한다.

기본 유형

만일 두 수를 입력받아 그를 합한 값을 출력하는 자동화 프로그램을 만들어보자.

기본 형태

```
declare /*선언부*/
        변수선언
begin /*알고리즘 기술부*/
    알고리즘 작성
end;
/       /*PL/SQL 종료*/
```

DECLARE 선언 전에 기본 설정

set serveroutput on

결과값을 콘솔(화면)에 표현되도록 하는 함수를 활성화(on)

accept 변수명

화면에 입력된 값을 변수에 저장

prompt '메세지'

화면에 '메세지'를 출력

DECLARE 선언부

필요한 변수명과 형식, 크기를 선언한다.

예를 들면 입력된 두 값을 합산하여 그 결과를 저장하는 변수가 필요할 때..

```
p_sum        number(10);
```

BEGIN ~ END 작성부문

미리 선언된 두개의 변수를 입력받아서 합산하여 그 결과를 p_sum에 입력

p_sum의 값을 화면(콘솔에 출력하고 종료)

```
p_sum := &입력변수1 + &입력변수2;

dbms_output.put_line ( '합은 : ' || p_sum || '입니다.');
```

예제

사원번호를 묻고 입력된 사원번호의 이름과 월급을 출력하는 프로시저를 작성하라.

```
accept

declare

begin

end;

/
```

예제

p_ename, p_sal 변수 선언

```
select ename, sal into p_ename, p_sal
from
where empno = &p_empno;
```

IF ~ THEN ~ ELSE

숫자를 입력하여 짝수 인지 홀수 인지 출력하시오.

```
 begin
  if mod(&변수명, 2) = 0
  then   dbms_output.put_line('짝수입니다.');
  else
          dbms_output.put_line('홀수입니다.');
  else if;
end;
```

심화 예제

사원번호를 입력하여 존재하는 사원이면 사원명과 급여를 출력하고 존재하지 않으면 '존재하지 않는 사원입니다.'라는 메시지를 출력하는 프로시저를 작성하시오.

작성은 이전 예제를 중심으로 작성한다.

IF ~ THEN ~ ELSE문을 작성하자.

힌트 : select count(*) into 변수명 from emp where empno = &변수명;

IF ~ THEN ~ ELSIF ~ ELSE

사원명을 입력받아 급여가 3000 이상이면 'A',

2000 이상 3000미만 이면 'B',

2000 미만 이면 'C'를 출력하는 프로시저를 작성하시오.

IF ~ THEN ~ ELSIF ~ ELSE ?

DECODE ?

IF ~ THEN ~ ELSIF ~ ELSE

```
DECLARE
p_ename emp.ename%type := upper('&p_ename');
P_sal emp.sal%type;
```

왜 p_name varchar2(20); 이 아닐까? /* 어떤 차이가 있을까? */

테이블에 이미 정의된 타입을 그대로 사용하고 싶으면..

%TYPE을 사용하자!

IF ~ THEN ~ ELSIF ~ ELSE

```
BEGIN 정의
SELECT sal INTO p_sal FROM emp
WHERE ename = p_ename;

IF p_sal >= 3000  THEN dbms_output.put_line ('A');
   ELSIF p_sal >= 2000 THEN dbms_output.put_line ('B');
   ELSE dbms_output.put_line ('C');
END IF;
```

IF ~ THEN ~ ELSIF ~ ELSE

BEGIN 정의

(1)

```
if var1 = 'a' then var2 := 'x';
elsif var1 = 'b' then var2 := 'y';
else var2 := 'z';
end if;
```

(2)

```
select decode(var1,'a','x','b','y','z') into var2 from dual;
```

LOOP 반복문

LOOP

~

EXIT WHEN 종료 조건;

END LOOP;

일단 한 번 수행 후 조건 비교!

EXIT WHEN이 누락되거나 잘못 기술하면 무한 루프를 수행한다.

구구단 중 2단을 출력해보자.

LOOP 반복문

```
declare
    p_count      number(10) := 0;
begin
    loop
        p_count := p_count + 1;
        dbms_output.put_line ( ' 2 * ' || p_count || ' = ' || 2*p_count);
        exit when p_count = 9;
    end loop;
```

WHILE ~ LOOP문

WHILE 종료 조건 LOOP

~

END LOOP

조건 검증을 먼저 후 수행(조건이 맞지 않으면 한번도 실행 안 할 수 있다.)

종료조건을 잘못 구성하면 무한루프를 수행한다.

구구단 중 2단을 구현해보자.

WHILE ~ LOOP문

```
declare
   p_count      number(10) := 0;
begin
      while p_count  < 9 loop
         p_count := p_count + 1;
         dbms_output.put_line ( ' 2 * ' || p_count || ' = ' || 2*p_count);
      end loop;
```

FOR ~ LOOP

for 수행 조건 loop

~

end loop;

예) for 1 in 1..9 loop ~

이전 반복문 보다 작성이 간단하여 무한루프가 수행될 가능성이 적다.

FOR ~ LOOP문

```
begin
    for i in 1..9 loop
        dbms_output.put_line ( ' 2 * ' || i || ' = ' || 2*i);
    end loop;
```

이중 FOR ~ LOOP

```
for 수행 조건 loop
    for 수행조건 loop
     ~
    end loop;
end loop;
```

구구단 전체를 수행하는 프로시저를 작성해보자.

이중 FOR ~ LOOP

```
begin
        for i in 1..9 loop
            for j in 1..9 loop
            dbms_output.put_line (  i  '*'  ||    j    || ' = ' || i*j);
            end loop;
        end loop;
end;
```

커서 (CURSOR)

다중행의 리턴값을 받아 한 데이터씩 처리하는 방법

대부분의 현업에서는 단일값 프로시저가 아닌 커서로 구현한다.

반복문(LOOP, WHILE, FOR)을 사용하여 단건 씩 처리한다.

커서를 먼저 선언하고

선언된 커서를 오픈하여

LOOP에서 처리한다.

커서를 종료한다.

커서 (CURSOR)

형식

```
declare
  필요 변수 선언;
  cursor 커서명 is
    select  ~ from ~ where ;
```

커서 (CURSOR)

형식

```
begin
  open 커서명;
  loop
      fetch  커서명 into 변수명,,,
      exit when 커서명%notfound;
      ~
   end loop;
  close 커서명;
```

방법1

부서를 입력받아 입력받은 부서번호를 통해 속한 사원명, 급여, 부서번호 출력하는 커서를 작성해보자.

부서명 콘솔에서 입력받자.

```
set serveroutput on

set verify off

accept p_deptno prompt '부서번호를 입력하시오.';
```

방법1-계속

```
declare
v_ename   emp.ename%type; /*변수선언*/
v_sal        emp.sal%type;
v_deptno emp.deptno%type;
cursor emp_cursor is     /* 커서 정의, 주기억장치에 값을 저장 */
      select ename, sal, deptno
      from emp
      where deptno = &p_deptno;
```

방법1-계속

```
begin
  open emp_cursor ; /* 커서 오픈 */
  loop
      fetch emp_cursor into v_ename, v_sal, v_deptno;
      exit when emp_cursor%notfound;
      dbms_output.put_line ( v_ename || '     ' || v_sal || '     ' || v_deptno);
   end loop;
  close emp_cursor;
end;
/
```

방법2

```
set serveroutput on
set verify off
accept p_deptno prompt '부서번호를 입력하시오.';
 declare
   cursor emp_cursor is     /* 커서 정의, 주기억장치에 값을 저장 */
       select ename, sal, deptno
       from emp
       where deptno = &p_deptno;
```

방법2-계속

```
begin
   for emp_record in emp_cursor
    loop
       dbms_output.put_line (emp_record.ename || '     '
                                || emp_record.sal || '     '
                                || emp_record.deptno);
    end loop;
end;
/
```

방법3

```
set serveroutput on
set verify off
accept p_deptno prompt '부서번호를 입력하시오.';
 begin
    for emp_record in
       (select ename, sal, deptno
       from emp
       where deptno = &p_deptno)
```

방법3-계속

```
loop
        dbms_output.put_line (emp_record.ename || '      '
                              || emp_record.sal || '      '
                              || emp_record.deptno);
    end loop;
end;
/
```